

Quiz 2 (Version 4)

CS 320: Concepts of Programming Languages

September 30, 2026

Name: Ankush Das

BUID:

- You will have approximately 35 minutes to complete this quiz. Make sure to read every question; some are easier than others.
- Make it very clear what your final solution for each problem is (e.g., by surrounding it in a box). We reserve the right to mark off points if we cannot tell what your final solution is.
- You must show your work on all problems unless otherwise specified. A solution without work will be considered incorrect (and will be investigated for potential academic misconduct).
- We will not look at any work on the pages marked *"This page is intentionally left blank."*. You should use these pages for scratch work.

1 OCaml Programming

(10 points) Implement the function

```
val idx_of_kth : int -> 'a -> 'a list -> int
```

so that `idx_of_kth k a l` is the index of the k th appearance (0-indexed) of `a` in `l`. If `l` has fewer than $k + 1$ appearances of `a`, then `idx_of_kth k a l` should be `-1`. The behavior of the function is undefined if `k` is negative.

```
let _ = assert (idx_of_kth 1 "a" ["a";"b";"a";"a"] = 2)
let _ = assert (idx_of_kth 0 3 [1;2;3] = 2)
let _ = assert (idx_of_kth 2 1 [1;2;1] = -1)
```

Your solution must be tail recursive. The only standard library functions you may use are arithmetic operators (e.g., `(+)` and `(-)`), comparisons (e.g., `(=)` and `(<=)`), list constructors (i.e., `[]` and `(::)`), and `List.rev`.

Solution:

```
let idx_of_kth k a l =
  let rec helper k a l ind =
    if k < 0 then ind-1
    else
      match l with
      | [] -> -1
      | x::xs -> if a = x then helper (k-1) a xs (ind+1) else helper k a xs (ind+1)
  in
  helper k a l 0
```

2 Typing Judgments

(10 points) Determine a minimal *monomorphic* context Γ and type τ such that the following typing judgment holds in OCaml. Here, monomorphic means there should be no type parameters in Γ (this means the judgment should also hold in CS320Caml). Justify your answer.

$$\Gamma \vdash \text{if } x \ y \ z \ \&\& \ z \ \text{then } 0 \ \text{else } y \ z : \tau$$

Solution:

Since `&&` occurs in the expression, `x y z` and `z` must be booleans.

Because the if-then-else expression should have the same type across both branches, and `y z` must be an integer.

Since `y` is being applied to `z` (a boolean), it must have type `bool -> int`.

Since `x` is being applied to `y` and `z` and its return value is a conditional, `x` must return a boolean. And we already know `y`'s type is `bool -> int`, which must be the first argument type of `x`. And, the second argument of `x` is `z`, so its second argument type must be a boolean.

So, the type of `x` must be `(bool -> int) -> bool -> bool`.

Hence, the minimal context Γ should be

$$\Gamma = \{x : (\text{bool} \rightarrow \text{int}) \rightarrow \text{bool} \rightarrow \text{bool}, y : \text{bool} \rightarrow \text{int}, z : \text{bool}\}$$

And τ must be `int`.

3 Expressions of Types

(10 points) Determine an expression of the following polymorphic type. That is, if you typed your expression into UTop, its type would be reported as the following type. You may only use basic OCaml constructs like anonymous functions, applications, and list constructors. In particular, you may not use recursion, type annotations, or assertions. Justify your answer.

```
(( 'a -> 'a) -> 'b) -> 'b
```

Solution:

```
fun x -> x (fun y -> y)
```

Clearly, there is one `->` on the outside, so the expression must be a function that takes one argument and returns an expression of type `'b`.

Thus, it must have the form

```
fun x -> ???
```

The argument `x` has type `('a -> 'a) -> 'b`, and `???` has type `'b`. Now, `x` is a function that takes an argument of type `'a -> 'a`, and returns `'b`. So, if we find a function, say `f` of type `'a -> 'a`, and call `x` with `f` as argument, we are all done, and our `???` can be `x f`.

So, what should `f` be? It is a function that takes `'a` as argument, say `y`, and returns `'a`. So, it must be the identity function. So, `f` can simply be `fun y -> y`.

This page is intentionally left blank.